ZipperGen Workflow Development and Deployment Manual

Concepts, commands, configuration, runtime, and operations

The ZipperGen Authors

Manual edition: 8 August 2026

Abstract

This manual is what you need after the tutorial. It covers the project's files, the workflow language, inspection and validation, running and testing, models and connectors, deployment and operations, and what to do when something is wrong.

It assumes you can use a terminal and an editor. It does not assume you are a Python or operations engineer, and it does not assume you use a coding agent, though it says where one helps.

Start with the tutorial

`docs/first-workflow.pdf` builds one small application end to end in about seven pages. This manual explains the parts it passes over. If you have not read it, read it first. Most of what follows will then be lookup rather than learning.

The promise

The workflow is always ordinary Python. A coding agent may write or change it, but ZipperGen does the deterministic work: loading it, projecting it for every participant, checking that the result is well formed, running it durably, and deploying it. Opaque model output never becomes a deployed application without passing through visible code you can read.

Contents

1 The project

A ZipperGen project is a directory. `zg init` creates one:

File	In git	Holds
`specification.md`	yes	what the workflow should do, in prose
`workflow.py`	yes	the coordination, as Python
`zippergen.toml`	yes	models, assistants, connectors, entry point, no secrets
`AGENTS.md`	yes	points a coding agent at `zg skill`
`CLAUDE.md`	yes	points Claude Code at `AGENTS.md`
`ZIPPERGEN_HOME`	no	credentials, runs, deployments

The split that matters is the last row. Everything above it travels with the project. The last row describes one machine. A colleague who clones the repository gets the workflow, the specification and the configuration, and supplies their own keys.

1.1 `zippergen.toml`

```
schema_version = 1
name = "email-approval"
specification_file = "specification.md"
workflow_entry = "workflow.py:email_approval"

[models.configurations."openai-gpt-4o"]
provider = "openai"
model = "gpt-4o"
spec = "openai:gpt-4o"

[models.assignments]
default = "mock"

[models.assignments.lifelines]
Writer = "openai-gpt-4o"

[connectors.configurations."approval-chat"]
provider = "telegram"
kind = "telegram"
chat_id = "12345678"

[connectors.assignments.lifelines]
User = "approval-chat"
```

It is an ordinary file. Edit it in an editor, or let a coding agent edit it. Nothing in ZipperGen owns it. `workflow_entry` is why most commands do not need you to name the workflow.

Named configurations exist so several participants can share one and it changes in one place. `configurations` says what a thing is, and `assignments` says who uses it.

1.2 What stays on one machine

API keys	environment variables, or `ZIPPERGEN_HOME`
Telegram bot token	typed once, never an argument
Google credentials	`zg connector authorize` / `accept`
Local model endpoints	site configuration
Runs and deployments	`ZIPPERGEN_HOME`

2 Writing workflows

A workflow is a Python function decorated with `@workflow`. Its body is read, rewritten, and turned into an intermediate representation, so it looks like Python and reads like a protocol, but it is not executed line by line as written.

2.1 Participants and messages

```
User = Lifeline("User")
Writer = Lifeline("Writer")

User(message) >> Writer(message)
```

The left side sends, the right receives. The names in brackets say which variables carry the value at each end. They need not match.

A self-send (`A(x) >> A(y)`) is a local rename, not a message.

2.2 Actions

An action is work one participant does.

`@llm`	a model call with a prompt and declared outputs
`@pure`	ordinary Python, output name and type from the return annotation
`@human`	stops and asks a person: `confirm`, `select`, `input`
`@assistant`	runs a coding agent against a repository
`@planner`	asks a model to generate a sub-workflow, validated before it runs

```
@llm(
    system="You write short, friendly replies.",
    user="Reply to this message:\n\n{message}",
    parse="text",
    outputs=[("draft", str)],
)
def draft_reply(message: str): ...

Writer: draft = draft_reply(message)
```

Several actions by one participant can be grouped:

```
with LLM1:
    agreed = check_agreement(verdict, other_verdict)
    trials = inc_trials(trials)
```

2.3 Decisions

```
if approved @ User:
    User: result = sent(draft)
else:
    User: result = discarded()
```

The `@ User` names the participant who evaluates the condition. It is required, and it is the most important annotation in the language: it decides who broadcasts the outcome and who merely receives it.

Loops work the same way:

```
while (not agreed and trials < MAX_ROUNDS) @ LLM1:
    ...
```

Parenthesise compound conditions

`@` binds more tightly than `not`, `and` and `or`. Write `if (not agreed) @ LLM1:`, not `if not agreed @ LLM1:`.

2.4 Parallel regions and co-regions

Independent branches that may interleave:

```
with parallel:
    with branch:
        Orchestrator(candidate) >> TestRunner(candidate)
        TestRunner: (test_status,) = run_tests(candidate)
        TestRunner(test_status) >> Committer(test_status)
    with branch:
        Orchestrator(candidate) >> Security(candidate)
        Security: (security_status,) = scan_security(candidate)
        Security(security_status) >> Committer(security_status)
```

A co-region is weaker: messages that may arrive in any order.

```
with coregion:
    Analyst_A(a) >> Collector(a_report)
    Analyst_B(b) >> Collector(b_report)
```

2.5 Inputs and outputs

```
def email_approval() -> int:
    ...
    return handled @ User
```

An input belongs to the participant that supplies it. The return says who holds the result. `zg validate` lists all declared workflow inputs. If an interactive run still needs any values, it introduces them under a `Workflow inputs` heading and shows declared defaults.

3 Inspecting and validating

3.1 `zg validate`

```
zg validate
```

Loads the module, projects the protocol for every participant, checks the deployment declaration, and renders the result back. If it passes, the workflow is well formed and the deadlock-freedom guarantee applies to it.

This is the check to run after every change. It is fast and needs nothing configured.

3.2 `zg show`

`zg show`	the global protocol
`zg show --communications`	messages only, no action bodies
`zg show --agent NAME`	one participant's local program
`zg show --detail full`	everything, including action definitions
`zg show --format json`	structured, for tooling

`--agent` is the interesting one. It shows what a participant actually runs after projection, which is not what you wrote:

```
@role('Writer')
def email_approval__Writer() -> None:
    while recv_decision('User'):
        message = recv('User')
        draft = draft_reply(message)
        send('User', draft)
```

The `Writer` is told each round whether to continue, because it has work inside the loop. It has no `if`, because it takes no part in the `User`'s decision. A participant's local program mentions only what concerns it, which is why it cannot wait for a message the decider chose not to send.

Use this when a workflow behaves in a way you did not expect. The global view says what you wrote. The local view says what will run.

4 Running and testing

4.1 One verb

`zg run`	execute once, nothing is kept
`zg run --durable`	record every step, so it can be resumed
`zg run --resume`	continue the most recent unfinished run
`zg run --resume --run-id ID`	continue a particular one

A plain run leaves nothing behind, so `zg deploy status` afterwards finds nothing. That is deliberate: throwaway runs should not accumulate. When you want to come back to a run, to resume it, inspect it, or answer a human action later, use `--durable`.

Missing inputs are asked for in a terminal. In a script, supply them:

```
zg run --input message="Could we move our meeting to Thursday?" --yes
```

4.2 One configuration pattern

Models, coding assistants, and connectors use the same small grammar:

```
zg TYPE configure NAME PROVIDER_OR_SPEC
zg TYPE assign TARGET NAME
zg TYPE check [NAME]
zg TYPE remove NAME
```

An external-service connector uses `connector bind REQUIREMENT NAME` instead of `assign`. The provider or backend is an attribute of the named configuration, not another object to manage. In the examples, `approval-chat` is a name chosen by the user and `telegram` is the provider. Bare `zg model`, `zg assistant`, and `zg connector` show each family. `zg config` shows the complete effective result.

In an interactive terminal, required values may be left out. ZipperGen asks for them and shows known targets and configurations when useful. Thus `zg model configure`, `zg assistant configure`, and `zg connector configure` are guided commands. A script or coding agent must pass the required values explicitly, so it cannot wait for an unseen question. Guided model setup asks for provider and model separately. Explicit commands use the compact `PROVIDER:MODEL` form. The Python API is always explicit and never prompts.

4.3 Choosing a model

For a durable project choice, create a named configuration and assign it:

```
zg model configure writer openai:gpt-4o-mini
zg model assign Writer writer
zg model
```

If the provider needs an API key and none is available, the configuration command offers a hidden prompt. The key is saved in private storage on this computer. It is never written to `zippergen.toml`. One provider key may serve several named model configurations. You may instead set the normal environment variable before running the command.

`--llm mock`	a placeholder for every action, no key needed
`--llm openai:gpt-4o-mini`	a real provider
`--llm scripted:replies.json`	recorded answers, see below
`--llm-for Writer=openai:gpt-4o`	override one participant or action

Without `--llm`, the assignments in `zippergen.toml` apply. The same resolution is used for plain runs, durable runs and deployments. `--llm` is a global one-command override, so it replaces participant and action assignments. Use `--llm-for` for a narrower temporary override. A deployment stores the resolved model specs in its profile; deploy again after changing the project assignments.

`zg model check` checks the selected model credentials without printing them. Add `--live` only when you want to make a small real provider request. Use `zg model unassign TARGET` before removing a configuration that is still referenced.

4.4 Choosing a coding assistant

An `@assistant` action runs Codex or Claude against a repository. Choose the CLI with the same named configuration and assignment pattern:

```
zg assistant configure coding-agent codex
zg assistant assign Maintainer coding-agent
zg assistant check
```

Assign a participant to cover all of its assistant actions. Use an exact target such as `Maintainer.update_release_notes` when one action needs a different backend. Plain runs, durable runs, and deployments use the same routing. A deployment stores the resolved backends in its profile, so deploy again after changing an assignment.

Backend choice and action permissions are separate. The named configuration chooses Codex or Claude. The `@assistant` declaration still owns its fixed instructions and its `access`, `external_tools`, and `shell` policies. `zg config` shows all of them together. `zg assistant check` verifies that each selected CLI is installed and supports every safety option that ZipperGen relies on. It does not log in. Codex and Claude continue to manage their own authentication.

4.5 Deterministic testing

`mock` answers every action the same way, so a workflow driven by it takes one path and no other. In a consensus protocol that means immediate agreement. Everything behind a decision is unreachable.

Scripted responses fix that:

```
{
  "LLM1.assess":     {"verdict": "yes", "reason": "unmoved"},
  "LLM2.assess":     {"verdict": "no",  "reason": "unmoved"},
  "LLM1.reconsider": {"verdict": "yes", "reason": "unmoved"},
  "LLM2.reconsider": {"verdict": "no",  "reason": "unmoved"}
}
```

```
zg run --llm scripted:never-agree.json
```

Keys are `action` or `Participant.action`. The more specific one wins. Values follow one rule worth learning:

`{...}`	a constant: every call is answered the same way
`[{...}, {...}]`	a finite sequence: exactly these calls are expected

Running past the end of a list is an error, not a silent repeat. That is the point: if a change makes an action run more often than the script expects, the run fails instead of passing quietly.

> **Use it for branches, not for prompts**
>
> Scripted responses exercise *coordination*: this reviewer disagrees, that loop runs three times, this branch is taken. They say nothing about whether a real model would produce good text. Both matter, and they are different questions.

4.6 Durable runs

A durable run records coordination state and completed external-action results in its own SQLite store. An ordinarily interrupted run resumes where it stopped. A crash after an external effect succeeds but before its result is recorded can repeat that effect, so irreversible connectors should use idempotency keys.

```
zg run --durable --llm openai:gpt-4o-mini
# Ctrl-C
zg inspect --agent Writer
zg run --resume
```

To watch a live durable run from another terminal, use:

```
zg inspect --watch --agent Writer
```

The display refreshes in place once per second. Ctrl-C closes the display and does not interrupt the run. Use `--interval SECONDS` when a slower refresh is more appropriate.

`running`	executing now
`waiting`	stopped at a human action
`interrupted`	you pressed Ctrl-C
`failed`	a participant raised an error
`done`	finished, with a result

`interrupted` and `failed` can be resumed. `done` cannot.

5 Models, people, and services

5.1 Models

For development, an environment variable is enough:

```
export OPENAI_API_KEY=sk-...
zg run --llm openai:gpt-4o-mini
```

`openai:MODEL`	`OPENAI_API_KEY`
`anthropic:MODEL` (or `claude:`)	`ANTHROPIC_API_KEY`
`mistral:MODEL`	`MISTRAL_API_KEY`
`ollama:MODEL` (or `local:`)	`OLLAMA_BASE_URL`
`mock`	none

To fix the choice in the project rather than the command line, put it in `zippergen.toml` under `models`. The guided command does this and can also collect the key privately:

```
zg model configure
Model configuration name: writer
Available model providers: openai, anthropic, mistral, local, scripted
Model provider: openai
Model name: gpt-4o-mini
OPENAI_API_KEY (input hidden. Press Enter to configure later):
```

The name and model are project configuration. The key is a site credential. They are deliberately stored in different places.

5.2 Asking a person

A `@human` action stops and waits. Where the question appears depends on configuration:

nothing configured	the terminal running the workflow
a Telegram connector, assigned	a message with buttons

The terminal is right while developing and wrong for a deployment nobody is watching. Two commands, doing two different things:

```
zg connector configure approval-chat telegram
zg connector assign User approval-chat
```

The first says *which* chat. The second says *who* is asked there: a participant, or one action with `User.approve_reply`.

The bot token is typed without echo and stays on this machine. The chat id goes in `zippergen.toml` and is committed: a colleague gets the chat, and supplies their own token.

5.3 Reading and writing external services

A workflow declares what it needs:

```
zippergen_connectors = (
    ConnectorRequirement(
        name="call-mailbox", kind="gmail",
        participant="Mailbox", access="read-write",
    ),
)
```

and the project says which one, binding it to that requirement by name:

```
zg connector configure inbox gmail \
    --query "is:unread in:inbox"
zg connector bind call-mailbox inbox
zg connector configure records google-sheets \
    --spreadsheet-id 1AbC... --tab Calls
zg connector bind call-records records
```

5.4 Authorizing Google

```
zg connector authorize google --scopes gmail.modify,spreadsheets
```

This opens a browser, and prints an encoded result rather than saving it. On the machine that will run the workflow:

```
zg connector accept google
```

which asks for the result without echo and stores the credential together with the scopes Google actually granted.

The two halves exist because a server usually has no browser. Authorize on your own computer, then accept on the server. The credential never appears in a command line, so it does not reach shell history.

Scopes are checked before deployment

Deploying refuses when the granted scopes do not cover what the workflow needs For example, a workflow that marks mail as read needs `gmail.modify`, not `gmail.readonly`. Re-run `authorize` with the missing scopes.

6 Deployment

6.1 Prepare, then start

```
zg deploy create --name production --no-start
zg deploy start production
```

These are deliberately separate. Preparing writes the bundle, a managed Python environment, and the service files, and reaches nothing. A deployment can be prepared today and started next week. Starting is about to make real calls, so every model, assistant CLI, and connector is probed first and a failure stops it.

After the first time, the deployment has a name and you use that:

```
zg deploy create production   # redeploy the current source
```

6.2 Operating one

`zg deploy`	every deployment on this machine
`zg deploy status NAME`	is it running, and where is it
`zg deploy logs NAME`	what it has been doing
`zg inspect NAME`	where each participant's local program is now
`zg inspect NAME --watch`	keep that position open and refresh it in place
`zg trace NAME`	recent protocol events
`zg tasks NAME`	decisions waiting for a person
`zg approve NAME`	answer one
`zg deploy check NAME`	readiness checks, in detail
`zg deploy stop` / `restart NAME`	move the service
`zg deploy compact NAME`	reclaim space, the service must be stopped
`zg deploy remove NAME`	delete it

6.3 What removal keeps

durable store, log	kept	the record of what actually ran
profile	kept	what produced them
secrets	**deleted**	must not be left behind
environment, bundle	deleted	rebuilt by deploying again
service files	deleted	regenerated

`--purge` keeps nothing, including the store. What survives lands in `ZIPPERGEN_HOME/trash/deployments/`, readable only by you. Nothing prunes it, so look occasionally.

6.4 Moving a project to a server

1. Clone the repository. The workflow, specification and `zippergen.toml` come with it.
2. `zg validate` confirms the module loads in that environment.
3. Supply what the machine needs: model keys in the environment, the Telegram token via `connector configure`, Google via `authorize` on your laptop and `accept` there.
4. `zg deploy --name production --no-start`, then `zg deploy start production`.

Nothing secret is in the repository, so step 3 is the whole difference between a clone and a running service.

7 Working with a coding agent

ZipperGen ships instructions for coding agents. `zg init` writes an `AGENTS.md` pointing at them and a `CLAUDE.md` that imports those same instructions for Claude Code. `zg skill` prints the full guidance.

```
zg skill                  # what an agent should follow
zg skill --agents-md      # a pointer file for an existing project
```

A reasonable division:

the agent	writes and changes the workflow and specification, validates, runs, inspects projections, prepares deployments
you	credentials, authorization, and starting production

That division is a convention, not a wall: an agent with a shell can run any command you can, and can read any file you can. Credentials are typed rather than passed as arguments, so they stay out of the agent's conversation, your shell history and the repository. But once saved they live in a file under `ZIPPERGEN_HOME` that your user can read.

That distinction is worth keeping straight. Entering a credential interactively protects it *at the moment of entry*. It is not a capability boundary. For a real one, run the agent under a different account, or in an environment that does not carry `ZIPPERGEN_HOME`. Nothing in ZipperGen can supply that, because a process running as you can do what you can do.

The specification is yours and the agent's

ZipperGen does not check that `workflow.py` implements `specification.md`, because it cannot: one is prose. There is no gate, no provenance, and no ceremony around it. Keep it accurate the way you keep a README accurate, and tell your agent to update it before changing behaviour, which the shipped skill does.

8 When something is wrong

8.1 The workflow will not load

`zg validate` reports the exception. Common causes: a missing import, an action used before it is defined, or a compound condition without parentheses (`if (not agreed) @ LLM1:`).

8.2 A participant seems to be waiting forever

Look at what it actually runs:

```
zg show --agent ThatParticipant
```

A projected program cannot wait for a message nobody sends, so if a participant is stuck the cause is usually an action that never returns, such as a model call without a timeout, or a human action nobody has answered. `zg tasks` lists the second kind.

8.3 Only one branch ever runs

You are almost certainly using `--llm mock`, which answers every action identically. Script the responses instead.

8.4 A deployment refuses to start

Starting probes every model, assistant CLI, and connector. The message names what failed. The usual causes are a missing API key on that machine, a Google authorization that does not cover the workflow's scopes, or a Telegram token that was never supplied there.

8.5 `zg deploy status` shows nothing after a run

A plain run keeps nothing. Use `--durable` for a run you want to come back to.

8.6 Connector configured but the question still appears in the terminal

Configuring says which chat. Assigning says who is asked there. Both are needed:

```
zg connector assign User approval-chat
```

9 Command reference

Project	
`init [NAME]`	create a project here
`skill`	print the coding-agent instructions
Inspect	
`validate`	load, project, and check
`show`	render the protocol or one participant
`snapshot` / `diff`	record and compare semantics
Run	
`run`	execute, with `--durable` and `--resume`
`status` / `trace` / `tasks` / `approve`	inspect and answer
Configure	
`config` / `config check`	show or check all effective configuration
`model configure NAME SPEC`	save one named model configuration
`model assign TARGET NAME`	route a participant or action
`model unassign TARGET` / `model remove NAME`	remove routing or an unused configuration
`assistant configure NAME BACKEND`	save Codex or Claude under a project name
`assistant assign TARGET NAME`	route an assistant participant or action
`assistant check [NAME]`	verify the selected CLI and required safety options
`assistant unassign TARGET` / `assistant remove NAME`	remove routing or an unused configuration
`connector configure NAME PROVIDER`	save one named configuration
`connector assign TARGET NAME`	route human actions
`connector bind REQUIREMENT NAME`	bind an external service
`connector unassign TARGET` / `connector unbind REQUIREMENT`	remove connector routing
`connector check [NAME]` / `connector remove NAME`	check or remove
`connector authorize google`	browser flow, prints a result
`connector accept google`	save that result here
`completion zsh|bash|fish`	print dynamic shell completion
Deploy	
`deploy [--name NAME] [--no-start]`	prepare, and start
`start` / `stop` / `restart`	move the service
`logs` / `doctor`	operate
`compact` / `remove`	reclaim, delete

Every command takes `--help`. Commands that act on a workflow default to the project's. Commands that act on a deployment default to the most recent.

`zippergen` and `zg` are the same program.

10 Where the guarantee comes from

Projection is syntax-directed. For each decision, every participant is one of three things:

decider	evaluates the condition and broadcasts the outcome
recipient	appears in a branch, receives the outcome and follows it
bystander	appears in neither branch, the decision is erased from its program

The `Writer` in the tutorial is a bystander. Its local program does not mention the approval, so it cannot wait on it or disagree about it.

The main theorem states that the projected local programs produce exactly the behaviours of the global workflow, up to the control messages projection introduces. Deadlock freedom follows for well-formed workflows. Both are machine-checked in Lean 4.

That is the trade the whole system makes: write one protocol, accept the language's restrictions, and get a distributed implementation that cannot deadlock, rather than writing each agent separately and hoping.